



Introduction

Any problem that asks us to find the top/smallest/frequent 'K' elements among a given set falls under this pattern.

The best data structure that comes to mind to keep track of 'K' elements is **Heap**. This pattern will make use of the Heap to solve multiple problems dealing with 'K' elements at a time from a set of given elements.

Let's jump onto our first problem to develop an understanding of this pattern.

[← Back](#)[✓ Completed](#)[Next →](#)[Solution Review: Problem Challenge 1](#)[Top 'K' Numbers \(easy\)](#)



Top 'K' Numbers (easy)

We'll cover the following ^

- Problem Statement
- Try it yourself
- Solution
 - Code
 - Time complexity
 - Space complexity

Problem Statement

Given an unsorted array of numbers, find the 'K' largest numbers in it.

Note: For a detailed discussion about different approaches to solve this problem, take a look at [Kth Smallest Number](#).

Example 1:

Input: [3, 1, 5, 12, 2, 11], K = 3
Output: [5, 12, 11]

Example 2:

Input: [5, 12, 11, -1, 12], K = 3
Output: [12, 11, 12]

Try it yourself

Try solving this question here:



```
1 import java.util.*;
2
3 class KlargestNumbers {
4
5     public static List<Integer> findKlargestNumbers(int[] nums, int k) {
6         // TODO: Write your code here
7         return new ArrayList<>();
8     }
9
10    public static void main(String[] args) {
11        List<Integer> result = KlargestNumbers.findKlargestNumbers(new int[] { 3, 1, 5, 12, 2, 11 }, 3);
12        System.out.println("Here are the top K numbers: " + result);
13    }
```



```

14     result = KlargestNumbers.findKlargestNumbers(new int[] { 5, 12, 11, -1, 12 }, 3);
15     System.out.println("Here are the top K numbers: " + result);
16 }
17 }
18

```



Run

Save

Reset



Solution

A brute force solution could be to sort the array and return the largest K numbers. The time complexity of such an algorithm will be $O(N * \log N)$ as we need to use a sorting algorithm like [Timsort](#) if we use Java's

`Collection.sort()`. Can we do better than that?

The best data structure that comes to mind to keep track of top 'K' elements is [Heap](#). Let's see if we can use a heap to find a better algorithm.

If we iterate through the array one element at a time and keep 'K' largest numbers in a heap such that each time we find a larger number than the smallest number in the heap, we do two things:

1. Take out the smallest number from the heap, and
2. Insert the larger number into the heap.

This will ensure that we always have 'K' largest numbers in the heap. The most efficient way to repeatedly find the smallest number among a set of numbers will be to use a min-heap. As we know, we can find the smallest number in a min-heap in constant time $O(1)$, since the smallest number is always at the root of the heap. Extracting the smallest number from a min-heap will take $O(\log N)$ (if the heap has 'N' elements) as the heap needs to readjust after the removal of an element.

Let's take Example-1 to go through each step of our algorithm.

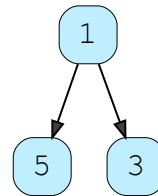


1. First, let's insert 'K' elements in the min-heap.
2. After the insertion, the heap will have three numbers [3, 1, 5] with '1' being the root as it is the smallest element.
3. We'll iterate through the remaining numbers and perform the above-mentioned two steps if we find a number larger than the root of the heap.
4. The 4th number is '12' which is larger than the root (which is '1'), so let's take out '1' and insert '12'. Now the heap will have [3, 5, 12] with '3' being the root as it is the smallest element.
5. The 5th number is '2' which is not bigger than the root of the heap ('3'), so we can skip this as we already have top three numbers in the heap.
6. The last number is '11' which is bigger than the root (which is '3'), so let's take out '3' and insert '11'. Finally, the heap has the largest three numbers: [5, 12, 11]

As discussed above, it will take us $O(\log K)$ to extract the minimum number from the min-heap. So the overall time complexity of our algorithm will be $O(K * \log K + (N - K) * \log K)$ since, first, we insert 'K' numbers in the heap and then iterate through the remaining numbers and at every step, in the worst case, we need to extract the minimum number and insert a new number in the heap. This algorithm is better than $O(N * \log N)$.

Here is the visual representation of our algorithm:

Given array:



Insert the first three numbers in the heap

1 of 4



Code

Here is what our algorithm will look like:



```
1 import java.util.*;
2
3 class KlargestNumbers {
4
5     public static List<Integer> findKlargestNumbers(int[] nums, int k) {
6         PriorityQueue<Integer> minHeap = new PriorityQueue<Integer>((n1, n2) -> n1 - n2);
7         // put first 'K' numbers in the min heap
8         for (int i = 0; i < k; i++)
9             minHeap.add(nums[i]);
10
11         // go through the remaining numbers of the array, if the number from the array is bigger than the
12         // top (smallest) number of the min-heap, remove the top number from heap and add the number from array
13         for (int i = k; i < nums.length; i++) {
14             if (nums[i] > minHeap.peek()) {
15                 minHeap.poll();
16                 minHeap.add(nums[i]);
17             }
18         }
19
20         // the heap has the top 'K' numbers, return them in a list
21         return new ArrayList<>(minHeap);
22     }
23
24     public static void main(String[] args) {
25         List<Integer> result = KlargestNumbers.findKlargestNumbers(new int[] { 3, 1, 5, 12, 2, 11 }, 3);
26         System.out.println("Here are the top K numbers: " + result);
27
28         result = KlargestNumbers.findKlargestNumbers(new int[] { 5, 12, 11, -1, 12 }, 3);
29         System.out.println("Here are the top K numbers: " + result);
30     }
31 }
32
```

Run

Save

Reset



Time complexity



As discussed above, the time complexity of this algorithm is $O(K * \log K + (N - K) * \log K)$, which is asymptotically equal to $O(N * \log K)$



Space complexity

The space complexity will be $O(K)$ since we need to store the top 'K' numbers in the heap.

[← Back](#)[✓ Mark As Completed](#)[Next →](#)[Introduction](#)[Kth Smallest Number \(easy\)](#)



Kth Smallest Number (easy)

We'll cover the following ^

- Problem Statement
- Try it yourself
- Solution
- Code
 - Time complexity
 - Space complexity
- An Alternate Approach

Problem Statement

Given an unsorted array of numbers, find Kth smallest number in it.

Please note that it is the Kth smallest number in the sorted order, not the Kth distinct element.

Note: For a detailed discussion about different approaches to solve this problem, take a look at [Kth Smallest Number](#).

Example 1:

Input: [1, 5, 12, 2, 11, 5], K = 3

Output: 5

Explanation: The 3rd smallest number is '5', as the first two smaller numbers are [1, 2].

Example 2:

Input: [1, 5, 12, 2, 11, 5], K = 4

Output: 5

Explanation: The 4th smallest number is '5', as the first three small numbers are [1, 2, 5].

Example 3:

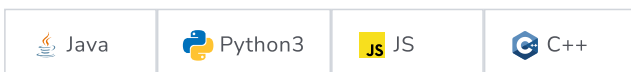
Input: [5, 12, 11, -1, 12], K = 3

Output: 11

Explanation: The 3rd smallest number is '11', as the first two small numbers are [5, -1].

Try it yourself

Try solving this question here:



```

1  import java.util.*;
2
3  class KthSmallestNumber {
4
5      public static int findKthSmallestNumber(int[] nums, int k) {
6          // TODO: Write your code here
7          return -1;
8      }
9
10     public static void main(String[] args) {
11         int result = KthSmallestNumber.findKthSmallestNumber(new int[] { 1, 5, 12, 2, 11, 5 }, 3);
12         System.out.println("Kth smallest number is: " + result);
13
14         // since there are two 5s in the input array, our 3rd and 4th smallest numbers should be a '5'
15         result = KthSmallestNumber.findKthSmallestNumber(new int[] { 1, 5, 12, 2, 11, 5 }, 4);
16         System.out.println("Kth smallest number is: " + result);
17
18         result = KthSmallestNumber.findKthSmallestNumber(new int[] { 5, 12, 11, -1, 12 }, 3);
19         System.out.println("Kth smallest number is: " + result);
20     }
21 }

```

Run

Save

Reset



Solution

This problem follows the [Top 'K' Numbers](#) pattern but has two differences:

1. Here we need to find the Kth **smallest** number, whereas in [Top 'K' Numbers](#) we were dealing with 'K' **largest** numbers.
2. In this problem, we need to find only one number (Kth smallest) compared to finding all 'K' largest numbers.

We can follow the same approach as discussed in the 'Top K Elements' problem. To handle the first difference mentioned above, we can use a max-heap instead of a min-heap. As we know, the root is the biggest element in the max heap. So, since we want to keep track of the 'K' smallest numbers, we can compare every number with the root while iterating through all numbers, and if it is smaller than the root, we'll take the root out and insert the smaller number.

Code

Here is what our algorithm will look like:

Java
 Python3
 C++
 JS

```

1  import java.util.*;
2
3  class KthSmallestNumber {
4
5      public static int findKthSmallestNumber(int[] nums, int k) {
6          PriorityQueue<Integer> maxHeap = new PriorityQueue<Integer>((n1, n2) -> n2 - n1);
7          // put first k numbers in the max heap
8          for (int i = 0; i < k; i++)
9              maxHeap.add(nums[i]);
10
11         // go through the remaining numbers of the array, if the number from the array is smaller than the
12         // top (biggest) number of the heap, remove the top number from heap and add the number from array
13         for (int i = k; i < nums.length; i++) {
14             if (nums[i] < maxHeap.peek()) {
15                 maxHeap.poll();
16                 maxHeap.add(nums[i]);
17             }
18         }
19         return maxHeap.poll();
20     }
21 }

```

```

16         maxHeap.add(nums[i]);
17     }
18 }
19
20 // the root of the heap has the Kth smallest number
21 return maxHeap.peek();
22 }
23
24 public static void main(String[] args) {
25     int result = KthSmallestNumber.findKthSmallestNumber(new int[] { 1, 5, 12, 2, 11, 5 }, 3);
26     System.out.println("Kth smallest number is: " + result);
27
28     // since there are two 5s in the input array, our 3rd and 4th smallest numbers should be a '5'
29     result = KthSmallestNumber.findKthSmallestNumber(new int[] { 1, 5, 12, 2, 11, 5 }, 4);
30     System.out.println("Kth smallest number is: " + result);
31
32     result = KthSmallestNumber.findKthSmallestNumber(new int[] { 5, 12, 11, -1, 12 }, 3);
33     System.out.println("Kth smallest number is: " + result); }
34 }
35

```

Run

Save

Reset



Time complexity

The time complexity of this algorithm is $O(K * \log K + (N - K) * \log K)$, which is asymptotically equal to $O(N * \log K)$

Space complexity

The space complexity will be $O(K)$ because we need to store 'K' smallest numbers in the heap.

An Alternate Approach

Alternatively, we can use a **Min Heap** to find the Kth smallest number. We can insert all the numbers in the min-heap and then extract the top 'K' numbers from the heap to find the Kth smallest number. Initializing the min-heap with all numbers will take $O(N)$ and extracting 'K' numbers will take $O(K \log N)$. Overall, the time complexity of this algorithm will be $O(N + K \log N)$ and the space complexity will be $O(N)$.

[< Back](#)
☒ Mark As Completed

[Next >](#)

Top 'K' Numbers (easy)

'K' Closest Points to the Origin (easy)



'K' Closest Points to the Origin (easy)

We'll cover the following ^

- Problem Statement
- Try it yourself
- Solution
- Code
 - Time complexity
 - Space complexity

Problem Statement

Given an array of points in a $2D$ plane, find 'K' closest points to the origin.

Example 1:

Input: points = [[1,2],[1,3]], K = 1

Output: [[1,2]]

Explanation: The Euclidean distance between (1, 2) and the origin is $\sqrt{5}$.

The Euclidean distance between (1, 3) and the origin is $\sqrt{10}$.

Since $\sqrt{5} < \sqrt{10}$, therefore (1, 2) is closer to the origin.

Example 2:

Input: point = [[1, 3], [3, 4], [2, -1]], K = 2

Output: [[1, 3], [2, -1]]

Try it yourself

Try solving this question here:



Java



Python3



JS



C++

```
1 import java.util.*;
2
3 class Point {
4     int x;
5     int y;
6
7     public Point(int x, int y) {
8         this.x = x;
9         this.y = y;
10    }
11
12    public int distFromOrigin() {
```



```

13     // ignoring sqrt
14     return (x * x) + (y * y);
15 }
16 }
17
18 class KClosestPointsToOrigin {
19
20     public static List<Point> findClosestPoints(Point[] points, int k) {
21         ArrayList<Point> result = new ArrayList<>();
22         // TODO: Write your code here
23         return result;
24     }
25
26     public static void main(String[] args) {
27         Point[] points = new Point[] { new Point(1, 3), new Point(3, 4), new Point(2, -1) };
28         List<Point> result = KClosestPointsToOrigin.findClosestPoints(points, 2);
29         System.out.print("Here are the k points closest the origin: ");
30         for (Point p : result)
31             System.out.print "[" + p.x + " , " + p.y + " ] ";
32     }
33 }
34

```

Run

Save

Reset



Solution

The [Euclidean distance](#) of a point P(x,y) from the origin can be calculated through the following formula:

$$\sqrt{x^2 + y^2}$$

This problem follows the [Top 'K' Numbers](#) pattern. The only difference in this problem is that we need to find the closest point (to the origin) as compared to finding the largest numbers.

Following a similar approach, we can use a **Max Heap** to find 'K' points closest to the origin. While iterating through all points, if a point (say 'P') is closer to the origin than the top point of the max-heap, we will remove that top point from the heap and add 'P' to always keep the closest points in the heap.

Code

Here is what our algorithm will look like:

```

1  import java.util.*;
2
3  class Point {
4      int x;
5      int y;
6
7      public Point(int x, int y) {
8          this.x = x;
9          this.y = y;
10     }
11
12     public int distFromOrigin() {
13         // ignoring sqrt
14         return (x * x) + (y * y);
15     }
16 }

```



```
17
18 class KClosestPointsToOrigin {
19
20     public static List<Point> findClosestPoints(Point[] points, int k) {
21         PriorityQueue<Point> maxHeap = new PriorityQueue<>((p1, p2) -> p2.distFromOrigin() - p1.distFromOrigin());
22         // put first 'k' points in the max heap
23         for (int i = 0; i < k; i++)
24             maxHeap.add(points[i]);
25
26         // go through the remaining points of the input array, if a point is closer to the origin than the top point
27         // of the max-heap, remove the top point from heap and add the point from the input array
28         for (int i = k; i < points.length; i++) {
29             if (points[i].distFromOrigin() < maxHeap.peek().distFromOrigin()) {
30                 maxHeap.poll();
31                 maxHeap.add(points[i]);
32             }
33         }
34
35         // the heap has 'k' points closest to the origin, return them in a list
36         return new ArrayList<>(maxHeap);
37     }
38
39     public static void main(String[] args) {
40         Point[] points = new Point[] { new Point(1, 3), new Point(3, 4), new Point(2, -1) };
41         List<Point> result = KClosestPointsToOrigin.findClosestPoints(points, 2);
42         System.out.print("Here are the k points closest the origin: ");
43         for (Point p : result)
44             System.out.print "[" + p.x + " , " + p.y + " ] ";
45     }
46 }
47
```

Run

Save

Reset



Time complexity

The time complexity of this algorithm is $(N * \log K)$ as we iterating all points and pushing them into the heap.

Space complexity

The space complexity will be $O(K)$ because we need to store 'K' point in the heap.

[< Back](#)☒ Mark As Completed[Next >](#)

Kth Smallest Number (easy)

Connect Ropes (easy)



Connect Ropes (easy)

We'll cover the following ^

- Problem Statement
- Try it yourself
- Solution
- Code
- Time complexity
- Space complexity

Problem Statement

Given 'N' ropes with different lengths, we need to connect these ropes into one big rope with minimum cost. The cost of connecting two ropes is equal to the sum of their lengths.

Example 1:

Input: [1, 3, 11, 5]

Output: 33

Explanation: First connect $1+3(=4)$, then $4+5(=9)$, and then $9+11(=20)$. So the total cost is 33 ($4+9+20$)

Example 2:

Input: [3, 4, 5, 6]

Output: 36

Explanation: First connect $3+4(=7)$, then $5+6(=11)$, $7+11(=18)$. Total cost is 36 ($7+11+18$)

Example 3:

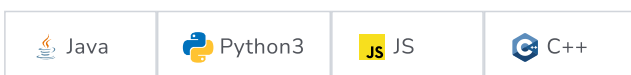
Input: [1, 3, 11, 5, 2]

Output: 42

Explanation: First connect $1+2(=3)$, then $3+3(=6)$, $6+5(=11)$, $11+11(=22)$. Total cost is 42 ($3+6+11+22$)

Try it yourself

Try solving this question here:



```
1 import java.util.*;
2
3 class ConnectRopes {
4
```



```

5   public static int minimumCostToConnectRopes(int[] ropeLengths) {
6       // TODO: Write your code here
7       return -1;
8   }
9
10  public static void main(String[] args) {
11      int result = ConnectRopes.minimumCostToConnectRopes(new int[] { 1, 3, 11, 5 });
12      System.out.println("Minimum cost to connect ropes: " + result);
13      result = ConnectRopes.minimumCostToConnectRopes(new int[] { 3, 4, 5, 6 });
14      System.out.println("Minimum cost to connect ropes: " + result);
15      result = ConnectRopes.minimumCostToConnectRopes(new int[] { 1, 3, 11, 5, 2 });
16      System.out.println("Minimum cost to connect ropes: " + result);
17  }
18  }
19

```

Run

Save

Reset



Solution

In this problem, following a greedy approach to connect the smallest ropes first will ensure the lowest cost. We can use a **Min Heap** to find the smallest ropes following a similar approach as discussed in [Kth Smallest Number](#). Once we connect two ropes, we need to insert the resultant rope back in the **Min Heap** so that we can connect it with the remaining ropes.

Code

Here is what our algorithm will look like:

Java

Python3

C++

JS

```

1  import java.util.*;
2
3  class ConnectRopes {
4
5      public static int minimumCostToConnectRopes(int[] ropeLengths) {
6          PriorityQueue<Integer> minHeap = new PriorityQueue<Integer>((n1, n2) -> n1 - n2);
7          // add all ropes to the min heap
8          for (int i = 0; i < ropeLengths.length; i++)
9              minHeap.add(ropeLengths[i]);
10
11         // go through the values of the heap, in each step take top (lowest) rope lengths from the min heap
12         // connect them and push the result back to the min heap.
13         // keep doing this until the heap is left with only one rope
14         int result = 0, temp = 0;
15         while (minHeap.size() > 1) {
16             temp = minHeap.poll() + minHeap.poll();
17             result += temp;
18             minHeap.add(temp);
19         }
20
21         return result;
22     }
23
24     public static void main(String[] args) {
25         int result = ConnectRopes.minimumCostToConnectRopes(new int[] { 1, 3, 11, 5 });
26         System.out.println("Minimum cost to connect ropes: " + result);
27         result = ConnectRopes.minimumCostToConnectRopes(new int[] { 3, 4, 5, 6 });
28         System.out.println("Minimum cost to connect ropes: " + result);
29         result = ConnectRopes.minimumCostToConnectRopes(new int[] { 1, 3, 11, 5, 2 });
30         System.out.println("Minimum cost to connect ropes: " + result);
31     }
32 }

```

```
32 }  
33
```

Run

Save

Reset



Time complexity#

Given 'N' ropes, we need $O(N * \log N)$ to insert all the ropes in the heap. In each step, while processing the heap, we take out two elements from the heap and insert one. This means we will have a total of 'N' steps, having a total time complexity of $O(N * \log N)$.

Space complexity

The space complexity will be $O(N)$ because we need to store all the ropes in the heap.

[< Back](#)☒ Mark As Completed[Next >](#)['K' Closest Points to the Origin \(easy\)](#)[Top 'K' Frequent Numbers \(medium\)](#)



Top 'K' Frequent Numbers (medium)

We'll cover the following ^

- Problem Statement
- Try it yourself
- Solution
- Code
 - Time complexity
 - Space complexity

Problem Statement

Given an unsorted array of numbers, find the top 'K' frequently occurring numbers in it.

Example 1:

Input: [1, 3, 5, 12, 11, 12, 11], K = 2
Output: [12, 11]
Explanation: Both '11' and '12' appeared twice.

Example 2:

Input: [5, 12, 11, 3, 11], K = 2
Output: [11, 5] or [11, 12] or [11, 3]
Explanation: Only '11' appeared twice, all other numbers appeared once.

Try it yourself

Try solving this question here:

Java

Python3

JS

C++

```
1 import java.util.*;
2
3 class TopKFrequentNumbers {
4
5     public static List<Integer> findTopKFrequentNumbers(int[] nums, int k) {
6         List<Integer> topNumbers = new ArrayList<>(k);
7         // TODO: Write your code here
8         return topNumbers;
9     }
10
11     public static void main(String[] args) {
12         List<Integer> result = TopKFrequentNumbers.findTopKFrequentNumbers(new int[] { 1, 3, 5, 12, 11, 12, 11 }, 2);
13         System.out.println("Here are the K frequent numbers: " + result);
14     }
15 }
```

```

14
15     result = TopKFrequentNumbers.findTopKFrequentNumbers(new int[] { 5, 12, 11, 3, 11 }, 2);
16     System.out.println("Here are the K frequent numbers: " + result);
17 }
18 }
19

```

Run

Save

Reset



Solution

This problem follows [Top 'K' Numbers](#). The only difference is that in this problem, we need to find the most frequently occurring number compared to finding the largest numbers.

We can follow the same approach as discussed in the **Top K Elements** problem. However, in this problem, we first need to know the frequency of each number, for which we can use a **HashMap**. Once we have the frequency map, we can use a **Min Heap** to find the 'K' most frequently occurring number. In the **Min Heap**, instead of comparing numbers we will compare their frequencies in order to get frequently occurring numbers

Code

Here is what our algorithm will look like:

Java

Python3

C++

JS

```

1  import java.util.*;
2
3  class TopKFrequentNumbers {
4
5      public static List<Integer> findTopKFrequentNumbers(int[] nums, int k) {
6          // find the frequency of each number
7          Map<Integer, Integer> numFrequencyMap = new HashMap<>();
8          for (int n : nums)
9              numFrequencyMap.put(n, numFrequencyMap.getOrDefault(n, 0) + 1);
10
11          PriorityQueue<Map.Entry<Integer, Integer>> minHeap = new PriorityQueue<Map.Entry<Integer, Integer>>(
12              (e1, e2) -> e1.getValue() - e2.getValue());
13
14          // go through all numbers of the numFrequencyMap and push them in the minHeap, which will have
15          // top k frequent numbers. If the heap size is more than k, we remove the smallest (top) number
16          for (Map.Entry<Integer, Integer> entry : numFrequencyMap.entrySet()) {
17              minHeap.add(entry);
18              if (minHeap.size() > k) {
19                  minHeap.poll();
20              }
21          }
22
23          // create a list of top k numbers
24          List<Integer> topNumbers = new ArrayList<>(k);
25          while (!minHeap.isEmpty()) {
26              topNumbers.add(minHeap.poll().getKey());
27          }
28          return topNumbers;
29      }
30
31      public static void main(String[] args) {
32          List<Integer> result = TopKFrequentNumbers.findTopKFrequentNumbers(new int[] { 1, 3, 5, 12, 11, 12, 1, 2 });
33          System.out.println("Here are the K frequent numbers: " + result);
34
35          result = TopKFrequentNumbers.findTopKFrequentNumbers(new int[] { 5, 12, 11, 3, 11 }, 2);
36          System.out.println("Here are the K frequent numbers: " + result);
37      }

```



```
38 }  
39
```

Run

Save

Reset

> Time complexity

The time complexity of the above algorithm is $O(N + N * \log K)$.

Space complexity

The space complexity will be $O(N)$. Even though we are storing only 'K' numbers in the heap. For the frequency map, however, we need to store all the 'N' numbers.

[< Back](#)[✓ Mark As Completed](#)[Next >](#)

Connect Ropes (easy)

Frequency Sort (medium)



Frequency Sort (medium)

We'll cover the following

- Problem Statement
- Try it yourself
- Solution
- Code
 - Time complexity
 - Space complexity

Problem Statement

Given a string, sort it based on the decreasing frequency of its characters.

Example 1:

Input: "Programming"

Output: "rrggmmPiano"

Explanation: 'r', 'g', and 'm' appeared twice, so they need to appear before any other character.

Example 2:

Input: "abcbab"

Output: "bbbaac"

Explanation: 'b' appeared three times, 'a' appeared twice, and 'c' appeared only once.

Try it yourself

Try solving this question here:

Java

Python3

JS

C++

```
1 import java.util.*;
2
3 class FrequencySort {
4
5     public static String sortCharacterByFrequency(String str) {
6         // TODO: Write your code here
7         return "";
8     }
9
10    public static void main(String[] args) {
11        String result = FrequencySort.sortCharacterByFrequency("Programming");
12        System.out.println("Here is the given string after sorting characters by frequency: " + result);
13    }
14}
```

```
14     result = frequencySort.sortCharacterByFrequency( abcdab );
15     System.out.println("Here is the given string after sorting characters by frequency: " + result);
16 }
17 }
```

Run

Save

Reset



Solution

This problem follows the **Top 'K' Elements** pattern, and shares similarities with [Top 'K' Frequent Numbers](#).

We can follow the same approach as discussed in the [Top 'K' Frequent Numbers](#) problem. First, we will find the frequencies of all characters, then use a max-heap to find the most occurring characters.

Code

Here is what our algorithm will look like:

Java

Python3

C++

JS

```
1  import java.util.*;
2
3  class FrequencySort {
4
5      public static String sortCharacterByFrequency(String str) {
6          // find the frequency of each character
7          Map<Character, Integer> characterFrequencyMap = new HashMap<>();
8          for (char chr : str.toCharArray()) {
9              characterFrequencyMap.put(chr, characterFrequencyMap.getOrDefault(chr, 0) + 1);
10         }
11
12         PriorityQueue<Map.Entry<Character, Integer>> maxHeap = new PriorityQueue<Map.Entry<Character, Integer>>((
13             (e1, e2) -> e2.getValue() - e1.getValue());
14
15         // add all characters to the max heap
16         maxHeap.addAll(characterFrequencyMap.entrySet());
17
18         // build a string, appending the most occurring characters first
19         StringBuilder sortedString = new StringBuilder(str.length());
20         while (!maxHeap.isEmpty()) {
21             Map.Entry<Character, Integer> entry = maxHeap.poll();
22             for (int i = 0; i < entry.getValue(); i++)
23                 sortedString.append(entry.getKey());
24         }
25         return sortedString.toString();
26     }
27
28     public static void main(String[] args) {
29         String result = FrequencySort.sortCharacterByFrequency("Programming");
30         System.out.println("Here is the given string after sorting characters by frequency: " + result);
31
32         result = FrequencySort.sortCharacterByFrequency("abcdab");
33         System.out.println("Here is the given string after sorting characters by frequency: " + result);
34     }
35 }
36
```

Run

Save

Reset



Time complexity

The time complexity of the above algorithm is $O(D * \log D)$ where 'D' is the number of distinct characters in the input string. This means, in the worst case, when all characters are unique the time complexity of the algorithm will be $O(N * \log N)$ where 'N' is the total number of characters in the string.



Space complexity

The space complexity will be $O(N)$, as in the worst case, we need to store all the 'N' characters in the HashMap.

[← Back](#)[☑ Mark As Completed](#)[Next →](#)[Top 'K' Frequent Numbers \(medium\)](#)[Kth Largest Number in a Stream \(medium\)](#)



Kth Largest Number in a Stream (medium)

We'll cover the following ^

- Problem Statement
- Try it yourself
- Solution
- Code
 - Time complexity
 - Space complexity

Problem Statement

Design a class to efficiently find the Kth largest element in a stream of numbers.

The class should have the following two things:

1. The constructor of the class should accept an integer array containing initial numbers from the stream and an integer 'K'.
2. The class should expose a function `add(int num)` which will store the given number and return the Kth largest number.

Example 1:

Input: [3, 1, 5, 12, 2, 11], K = 4
1. Calling `add(6)` should return '5'.
2. Calling `add(13)` should return '6'.
2. Calling `add(4)` should still return '6'.

Try it yourself

Try solving this question here:



Java



Python3



JS



C++

```
1 import java.util.*;
2
3 class KthLargestNumberInStream {
4
5     public KthLargestNumberInStream(int[] nums, int k) {
6         // TODO: Write your code here
7     }
8
9     public int add(int num) {
10        // TODO: Write your code here
```



```

11     return -1;
12 }
13
14 public static void main(String[] args) {
15     int[] input = new int[] { 3, 1, 5, 12, 2, 11 };
16     KthLargestNumberInStream kthLargestNumber = new KthLargestNumberInStream(input, 4);
17     System.out.println("4th largest number is: " + kthLargestNumber.add(6));
18     System.out.println("4th largest number is: " + kthLargestNumber.add(13));
19     System.out.println("4th largest number is: " + kthLargestNumber.add(4));
20 }
21 }

```

Run

Save

Reset



Solution

This problem follows the **Top 'K' Elements** pattern and shares similarities with [Kth Smallest number](#).

We can follow the same approach as discussed in the 'Kth Smallest number' problem. However, we will use a **Min Heap** (instead of a **Max Heap**) as we need to find the Kth largest number.

Code

Here is what our algorithm will look like:

Java

Python3

C++

JS

```

1  import java.util.*;
2
3  class KthLargestNumberInStream {
4      PriorityQueue<Integer> minHeap = new PriorityQueue<Integer>((n1, n2) -> n1 - n2);
5      final int k;
6
7      public KthLargestNumberInStream(int[] nums, int k) {
8          this.k = k;
9          // add the numbers in the min heap
10         for (int i = 0; i < nums.length; i++)
11             add(nums[i]);
12     }
13
14     public int add(int num) {
15         // add the new number in the min heap
16         minHeap.add(num);
17
18         // if heap has more than 'k' numbers, remove one number
19         if (minHeap.size() > this.k)
20             minHeap.poll();
21
22         // return the 'Kth largest number
23         return minHeap.peek();
24     }
25
26     public static void main(String[] args) {
27         int[] input = new int[] { 3, 1, 5, 12, 2, 11 };
28         KthLargestNumberInStream kthLargestNumber = new KthLargestNumberInStream(input, 4);
29         System.out.println("4th largest number is: " + kthLargestNumber.add(6));
30         System.out.println("4th largest number is: " + kthLargestNumber.add(13));
31         System.out.println("4th largest number is: " + kthLargestNumber.add(4));
32     }
33 }

```



[Run](#)[Save](#)[Reset](#)

Time complexity



The time complexity of the `add()` function will be $O(\log K)$ since we are inserting the new number in the heap.

Space complexity

The space complexity will be $O(K)$ for storing numbers in the heap.

[← Back](#)[✓ Mark As Completed](#)[Next →](#)[Frequency Sort \(medium\)](#)['K' Closest Numbers \(medium\)](#)



'K' Closest Numbers (medium)

We'll cover the following

- Problem Statement
- Try it yourself
- Solution
- Code
- Time complexity
- Space complexity
- Alternate Solution using Two Pointers
 - Time complexity
 - Space complexity

Problem Statement

Given a sorted number array and two integers 'K' and 'X', find 'K' closest numbers to 'X' in the array. Return the numbers in the sorted order. 'X' is not necessarily present in the array.

Example 1:

Input: [5, 6, 7, 8, 9], K = 3, X = 7
Output: [6, 7, 8]

Example 2:

Input: [2, 4, 5, 6, 9], K = 3, X = 6
Output: [4, 5, 6]

Example 3:

Input: [2, 4, 5, 6, 9], K = 3, X = 10
Output: [5, 6, 9]

Try it yourself

Try solving this question here:



```
1 import java.util.*;  
2  
3 class Entry {
```




```

4   int key;
5   int value;
6
7   public Entry(int key, int value) {
8       this.key = key;
9       this.value = value;
10  }
11 }
12
13 class KClosestElements {
14
15     public static List<Integer> findClosestElements(int[] arr, int K, Integer X) {
16         List<Integer> result = new ArrayList<>();
17         // TODO: Write your code here
18         return result;
19     }
20
21     public static void main(String[] args) {
22         List<Integer> result = KClosestElements.findClosestElements(new int[] { 5, 6, 7, 8, 9 }, 3, 7);
23         System.out.println("'K' closest numbers to 'X' are: " + result);
24
25         result = KClosestElements.findClosestElements(new int[] { 2, 4, 5, 6, 9 }, 3, 6);
26         System.out.println("'K' closest numbers to 'X' are: " + result);
27
28         result = KClosestElements.findClosestElements(new int[] { 2, 4, 5, 6, 9 }, 3, 10);
29         System.out.println("'K' closest numbers to 'X' are: " + result);
30     }
31 }
32

```

Run

Save

Reset



Solution

This problem follows the [Top 'K' Numbers](#) pattern. The biggest difference in this problem is that we need to find the closest (to 'X') numbers compared to finding the overall largest numbers. Another difference is that the given array is sorted.

Utilizing a similar approach, we can find the numbers closest to 'X' through the following algorithm:

1. Since the array is sorted, we can first find the number closest to 'X' through **Binary Search**. Let's say that number is 'Y'.
2. The 'K' closest numbers to 'Y' will be adjacent to 'Y' in the array. We can search in both directions of 'Y' to find the closest numbers.
3. We can use a heap to efficiently search for the closest numbers. We will take 'K' numbers in both directions of 'Y' and push them in a **Min Heap** sorted by their absolute difference from 'X'. This will ensure that the numbers with the smallest difference from 'X' (i.e., closest to 'X') can be extracted easily from the **Min Heap**.
4. Finally, we will extract the top 'K' numbers from the **Min Heap** to find the required numbers.

Code

Here is what our algorithm will look like:

```

1  import java.util.*;
2
3  class Entry {
4      int key;

```



```
5    int value;
6
7    public Entry(int key, int value) {
8        this.key = key;
9        this.value = value;
10   }
11 }
12
13 class KClosestElements {
14
15     public static List<Integer> findClosestElements(int[] arr, int K, Integer X) {
16         int index = binarySearch(arr, X);
17         int low = index - K, high = index + K;
18         low = Math.max(low, 0); // 'low' should not be less than zero
19         high = Math.min(high, arr.length - 1); // 'high' should not be greater the size of the array
20
21         PriorityQueue<Entry> minHeap = new PriorityQueue<>((n1, n2) -> n1.key - n2.key);
22         // add all candidate elements to the min heap, sorted by their absolute difference from 'X'
23         for (int i = low; i <= high; i++)
24             minHeap.add(new Entry(Math.abs(arr[i] - X), i));
25
26         // we need the top 'K' elements having smallest difference from 'X'
27         List<Integer> result = new ArrayList<>();
28         for (int i = 0; i < K; i++)
29             result.add(arr[minHeap.poll().value]);
30
31         Collections.sort(result);
32         return result;
33     }
34
35     private static int binarySearch(int[] arr, int target) {
36         int low = 0;
37         int high = arr.length - 1;
38         while (low <= high) {
39             int mid = low + (high - low) / 2;
40             if (arr[mid] == target)
41                 return mid;
42             if (arr[mid] < target) {
43                 low = mid + 1;
44             } else {
45                 high = mid - 1;
46             }
47         }
48         if (low > 0) {
49             return low - 1;
50         }
51         return low;
52     }
53
54     public static void main(String[] args) {
55         List<Integer> result = KClosestElements.findClosestElements(new int[] { 5, 6, 7, 8, 9 }, 3, 7);
56         System.out.println("'K' closest numbers to 'X' are: " + result);
57
58         result = KClosestElements.findClosestElements(new int[] { 2, 4, 5, 6, 9 }, 3, 6);
59         System.out.println("'K' closest numbers to 'X' are: " + result);
60
61         result = KClosestElements.findClosestElements(new int[] { 2, 4, 5, 6, 9 }, 3, 10);
62         System.out.println("'K' closest numbers to 'X' are: " + result);
63     }
64 }
65
```

Run

Save

Reset



Time complexity

The time complexity of the above algorithm is $O(\log N + K * \log K)$. We need $O(\log N)$ for Binary Search and $O(K * \log K)$ to insert the numbers in the **Min Heap**, as well as to sort the output array.

Space complexity

The space complexity will be $O(K)$, as we need to put a maximum of $2K$ numbers in the heap.

Alternate Solution using Two Pointers

After finding the number closest to 'X' through **Binary Search**, we can use the **Two Pointers** approach to find the 'K' closest numbers. Let's say the closest number is 'Y'. We can have a **left** pointer to move back from 'Y' and a **right** pointer to move forward from 'Y'. At any stage, whichever number pointed out by the **left** or the **right** pointer gives the smaller difference from 'X' will be added to our result list.

To keep the resultant list sorted we can use a **Queue**. So whenever we take the number pointed out by the **left** pointer, we will append it at the beginning of the list and whenever we take the number pointed out by the **right** pointer we will append it at the end of the list.

Here is what our algorithm will look like:



```

1  import java.util.*;
2
3  class KClosestElements {
4
5      public static List<Integer> findClosestElements(int[] arr, int K, Integer X) {
6          List<Integer> result = new LinkedList<>();
7          int index = binarySearch(arr, X);
8          int leftPointer = index;
9          int rightPointer = index + 1;
10         for (int i = 0; i < K; i++) {
11             if (leftPointer >= 0 && rightPointer < arr.length) {
12                 int diff1 = Math.abs(X - arr[leftPointer]);
13                 int diff2 = Math.abs(X - arr[rightPointer]);
14                 if (diff1 <= diff2)
15                     result.add(0, arr[leftPointer--]); // append in the beginning
16                 else
17                     result.add(arr[rightPointer++]); // append at the end
18             } else if (leftPointer >= 0) {
19                 result.add(0, arr[leftPointer--]);
20             } else if (rightPointer < arr.length) {
21                 result.add(arr[rightPointer++]);
22             }
23         }
24         return result;
25     }
26
27     private static int binarySearch(int[] arr, int target) {
28         int low = 0;
29         int high = arr.length - 1;
30         while (low <= high) {
31             int mid = low + (high - low) / 2;
32             if (arr[mid] == target)
33                 return mid;
34             if (arr[mid] < target) {
35                 low = mid + 1;

```

```
36         } else {
37             high = mid - 1;
38         }
39     }
40     if (low > 0) {
41         return low - 1;
42     }
43     return low;
44 }
45
46 public static void main(String[] args) {
47     List<Integer> result = KClosestElements.findClosestElements(new int[] { 5, 6, 7, 8, 9 }, 3, 7);
48     System.out.println("'K' closest numbers to 'X' are: " + result);
49
50     result = KClosestElements.findClosestElements(new int[] { 2, 4, 5, 6, 9 }, 3, 6);
51     System.out.println("'K' closest numbers to 'X' are: " + result);
52
53     result = KClosestElements.findClosestElements(new int[] { 2, 4, 5, 6, 9 }, 3, 10);
54     System.out.println("'K' closest numbers to 'X' are: " + result);
55 }
56 }
57
```

Run

Save

Reset



Time complexity

The time complexity of the above algorithm is $O(\log N + K)$. We need $O(\log N)$ for Binary Search and $O(K)$ for finding the 'K' closest numbers using the two pointers.

Space complexity

If we ignoring the space required for the output list, the algorithm runs in constant space $O(1)$.

[< Back](#)☒ Mark As Completed[Next >](#)

Kth Largest Number in a Stream (medium)

Maximum Distinct Elements (medium)



Maximum Distinct Elements (medium)

We'll cover the following ^

- Problem Statement
- Try it yourself
- Solution
- Code
 - Time complexity
 - Space complexity

Problem Statement

Given an array of numbers and a number 'K', we need to remove 'K' numbers from the array such that we are left with maximum distinct numbers.

Example 1:

Input: [7, 3, 5, 8, 5, 3, 3], and K=2

Output: 3

Explanation: We can remove two occurrences of 3 to be left with 3 distinct numbers [7, 3, 8], we have to skip 5 because it is not distinct and appeared twice.

Another solution could be to remove one instance of '5' and '3' each to be left with three distinct numbers [7, 5, 8], in this case, we have to skip 3 because it appeared twice.

Example 2:

Input: [3, 5, 12, 11, 12], and K=3

Output: 2

Explanation: We can remove one occurrence of 12, after which all numbers will become distinct. Then we can delete any two numbers which will leave us 2 distinct numbers in the result.

Example 3:

Input: [1, 2, 3, 3, 3, 3, 4, 4, 5, 5, 5], and K=2

Output: 3

Explanation: We can remove one occurrence of '4' to get three distinct numbers.

Try it yourself

Try solving this question here:



Java

Python3

JS

C++

```

1  import java.util.*;
2
3  class MaximumDistinctElements {
4
5      public static int findMaximumDistinctElements(int[] nums, int k) {
6          // TODO: Write your code here
7          return -1;
8      }
9
10     public static void main(String[] args) {
11         int result = MaximumDistinctElements.findMaximumDistinctElements(new int[] { 7, 3, 5, 8, 5, 3, 3 }, 2);
12         System.out.println("Maximum distinct numbers after removing K numbers: " + result);
13
14         result = MaximumDistinctElements.findMaximumDistinctElements(new int[] { 3, 5, 12, 11, 12 }, 3);
15         System.out.println("Maximum distinct numbers after removing K numbers: " + result);
16
17         result = MaximumDistinctElements.findMaximumDistinctElements(new int[] { 1, 2, 3, 3, 3, 3, 4, 4, 5, 5, 5 }, 2);
18         System.out.println("Maximum distinct numbers after removing K numbers: " + result);
19     }
20 }

```

Run

Save

Reset

Solution

This problem follows the [Top 'K' Numbers](#) pattern, and shares similarities with [Top 'K' Frequent Numbers](#).

We can follow a similar approach as discussed in [Top 'K' Frequent Numbers](#) problem:

1. First, we will find the frequencies of all the numbers.
2. Then, push all numbers that are not distinct (i.e., have a frequency higher than one) in a **Min Heap** based on their frequencies. At the same time, we will keep a running count of all the distinct numbers.
3. Following a greedy approach, in a stepwise fashion, we will remove the least frequent number from the heap (i.e., the top element of the min-heap), and try to make it distinct. We will see if we can remove all occurrences of a number except one. If we can, we will increment our running count of distinct numbers. We have to also keep a count of how many removals we have done.
4. If after removing elements from the heap, we are still left with some deletions, we have to remove some distinct elements.

Code

Here is what our algorithm will look like:

Java

Python3

C++

JS

```

1  import java.util.*;
2
3  class MaximumDistinctElements {
4
5      public static int findMaximumDistinctElements(int[] nums, int k) {
6          int distinctElementsCount = 0;
7          if (nums.length <= k)
8              return distinctElementsCount;
9
10         // find the frequency of each number

```

Tr

```

11 Map<Integer, Integer> numFrequencyMap = new HashMap<>();
12 for (int i : nums)
13     numFrequencyMap.put(i, numFrequencyMap.getOrDefault(i, 0) + 1);
14
15 PriorityQueue<Map.Entry<Integer, Integer>> minHeap = new PriorityQueue<Map.Entry<Integer, Integer>>(
16     (e1, e2) -> e1.getValue() - e2.getValue());
17
18 // insert all numbers with frequency greater than '1' into the min-heap
19 for (Map.Entry<Integer, Integer> entry : numFrequencyMap.entrySet()) {
20     if (entry.getValue() == 1)
21         distinctElementsCount++;
22     else
23         minHeap.add(entry);
24 }
25
26 // following a greedy approach, try removing the least frequent numbers first from the min-heap
27 while (k > 0 && !minHeap.isEmpty()) {
28     Map.Entry<Integer, Integer> entry = minHeap.poll();
29     // to make an element distinct, we need to remove all of its occurrences except one
30     k -= entry.getValue() - 1;
31     if (k >= 0)
32         distinctElementsCount++;
33 }
34
35 // if k > 0, this means we have to remove some distinct numbers
36 if (k > 0)
37     distinctElementsCount -= k;
38
39 return distinctElementsCount;
40 }
41
42 public static void main(String[] args) {
43     int result = MaximumDistinctElements.findMaximumDistinctElements(new int[] { 7, 3, 5, 8, 5, 3, 3 }, 2);
44     System.out.println("Maximum distinct numbers after removing K numbers: " + result);
45
46     result = MaximumDistinctElements.findMaximumDistinctElements(new int[] { 3, 5, 12, 11, 12 }, 3);
47     System.out.println("Maximum distinct numbers after removing K numbers: " + result);
48
49     result = MaximumDistinctElements.findMaximumDistinctElements(new int[] { 1, 2, 3, 3, 3, 3, 4, 4, 5, 5, 5 }, 2);
50     System.out.println("Maximum distinct numbers after removing K numbers: " + result);
51 }
52

```

Run

Save

Reset



Time complexity

Since we will insert all numbers in a **HashMap** and a **Min Heap**, this will take $O(N * \log N)$ where 'N' is the total input numbers. While extracting numbers from the heap, in the worst case, we will need to take out 'K' numbers. This will happen when we have at least 'K' numbers with a frequency of two. Since the heap can have a maximum of 'N/2' numbers, therefore, extracting an element from the heap will take $O(\log N)$ and extracting 'K' numbers will take $O(K \log N)$. So overall, the time complexity of our algorithm will be $O(N * \log N + K \log N)$.

We can optimize the above algorithm and only push 'K' elements in the heap, as in the worst case we will be extracting 'K' elements from the heap. This optimization will reduce the overall time complexity to $O(N * \log K + K \log K)$.

Space complexity

The space complexity will be $O(N)$ as, in the worst case, we need to store all the 'N' characters in the **HashMap**.

← Back

☒ Mark As Completed

Next →

'K' Closest Numbers (medium)

Sum of Elements (medium)





Sum of Elements (medium)

We'll cover the following ^

- Problem Statement
- Try it yourself
- Solution
- Code
 - Time complexity
 - Space complexity
- Alternate Solution
 - Time complexity
 - Space complexity

Problem Statement

Given an array, find the sum of all numbers between the K1'th and K2'th smallest elements of that array.

Example 1:

Input: [1, 3, 12, 5, 15, 11], and K1=3, K2=6
Output: 23
Explanation: The 3rd smallest number is 5 and 6th smallest number 15. The sum of numbers coming between 5 and 15 is 23 (11+12).

Example 2:

Input: [3, 5, 8, 7], and K1=1, K2=4
Output: 12
Explanation: The sum of the numbers between the 1st smallest number (3) and the 4th smallest number (8) is 12 (5+7).

Try it yourself

Try solving this question here:



```
1 import java.util.*;
2
3 class SumOfElements {
4
5     public static int findSumOfElements(int[] nums, int k1, int k2) {
6         // TODO: Write your code here
7     }
8 }
```



```

6 // TODO: write your code here
7 return -1;
8 }
9
10 public static void main(String[] args) {
11     int result = SumOfElements.findSumOfElements(new int[] { 1, 3, 12, 5, 15, 11 }, 3, 6);
12     System.out.println("Sum of all numbers between k1 and k2 smallest numbers: " + result);
13
14     result = SumOfElements.findSumOfElements(new int[] { 3, 5, 8, 7 }, 1, 4);
15     System.out.println("Sum of all numbers between k1 and k2 smallest numbers: " + result);
16 }
17 }
18

```

Run

Save

Reset



Solution

This problem follows the [Top 'K' Numbers](#) pattern, and shares similarities with [Kth Smallest Number](#).

We can find the sum of all numbers coming between the K1'th and K2'th smallest numbers in the following steps:

1. First, insert all numbers in a min-heap.
2. Remove the first **k1** smallest numbers from the min-heap.
3. Now take the next **k2-k1-1** numbers out of the heap and add them. This sum will be our required output.

Code

Here is what our algorithm will look like:

Java

Python3

C++

JS

```

1 import java.util.*;
2
3 class SumOfElements {
4
5     public static int findSumOfElements(int[] nums, int k1, int k2) {
6         PriorityQueue<Integer> minHeap = new PriorityQueue<Integer>((n1, n2) -> n1 - n2);
7         // insert all numbers to the min heap
8         for (int i = 0; i < nums.length; i++)
9             minHeap.add(nums[i]);
10
11         // remove k1 small numbers from the min heap
12         for (int i = 0; i < k1; i++)
13             minHeap.poll();
14
15         int elementSum = 0;
16         // sum next k2-k1-1 numbers
17         for (int i = 0; i < k2 - k1 - 1; i++)
18             elementSum += minHeap.poll();
19
20         return elementSum;
21     }
22
23     public static void main(String[] args) {
24         int result = SumOfElements.findSumOfElements(new int[] { 1, 3, 12, 5, 15, 11 }, 3, 6);
25         System.out.println("Sum of all numbers between k1 and k2 smallest numbers: " + result);
26
27         result = SumOfElements.findSumOfElements(new int[] { 3, 5, 8, 7 }, 1, 4);
28         System.out.println("Sum of all numbers between k1 and k2 smallest numbers: " + result);
29     }
30 }

```

31

Run

Save

Reset



Time complexity

Since we need to put all the numbers in a min-heap, the time complexity of the above algorithm will be $O(N * \log N)$ where 'N' is the total input numbers.

Space complexity

The space complexity will be $O(N)$, as we need to store all the 'N' numbers in the heap.

Alternate Solution

We can iterate the array and use a max-heap to keep track of the top K_2 numbers. We can, then, add the top $K_2 - K_1 - 1$ numbers in the max-heap to find the sum of all numbers coming between the K_1 'th and K_2 'th smallest numbers. Here is what the algorithm will look like:

Java

Python3

C++

JS

```

1  import java.util.*;
2
3  class SumOfElements {
4
5      public static int findSumOfElements(int[] nums, int k1, int k2) {
6          PriorityQueue<Integer> maxHeap = new PriorityQueue<Integer>((n1, n2) -> n2 - n1);
7          // keep smallest k2 numbers in the max heap
8          for (int i = 0; i < nums.length; i++) {
9              if (i < k2 - 1) {
10                 maxHeap.add(nums[i]);
11             } else if (nums[i] < maxHeap.peek()) {
12                 maxHeap.poll(); // as we are interested only in the smallest k2 numbers
13                 maxHeap.add(nums[i]);
14             }
15         }
16
17         // get the sum of numbers between k1 and k2 indices
18         // these numbers will be at the top of the max heap
19         int elementSum = 0;
20         for (int i = 0; i < k2 - k1 - 1; i++)
21             elementSum += maxHeap.poll();
22
23         return elementSum;
24     }
25
26     public static void main(String[] args) {
27         int result = SumOfElements.findSumOfElements(new int[] { 1, 3, 12, 5, 15, 11 }, 3, 6);
28         System.out.println("Sum of all numbers between k1 and k2 smallest numbers: " + result);
29
30         result = SumOfElements.findSumOfElements(new int[] { 3, 5, 8, 7 }, 1, 4);
31         System.out.println("Sum of all numbers between k1 and k2 smallest numbers: " + result);
32     }
33 }

```

Run

Save

Reset



Time complexity

Since we need to put only the top K^2 numbers in the max-heap at any time, the time complexity of the above algorithm will be $O(N * \log K^2)$.



Space complexity

The space complexity will be $O(K^2)$, as we need to store the smallest ' K^2 ' numbers in the heap.

[< Back](#)[✔ Mark As Completed](#)[Next >](#)[Maximum Distinct Elements \(medium\)](#)[Rearrange String \(hard\)](#)



Rearrange String (hard)

We'll cover the following ^

- Problem Statement
- Try it yourself
- Solution
- Code
 - Time complexity
 - Space complexity

Problem Statement

Given a string, find if its letters can be rearranged in such a way that no two same characters come next to each other.

Example 1:

Input: "aappp"
Output: "papap"
Explanation: In "papap", none of the repeating characters come next to each other.

Example 2:

Input: "Programming"
Output: "rgmrGmPiano" or "gmringmrPoa" or "gmrPagimnor", etc.
Explanation: None of the repeating characters come next to each other.

Example 3:

Input: "aapa"
Output: ""
Explanation: In all arrangements of "aapa", atleast two 'a' will come together e.g., "apaa", "paaa".

Try it yourself

Try solving this question here:



```
1 import java.util.*;
2
3 class RearrangeString {
4
```



```

5   public static String rearrangeString(String str) {
6       // TODO: Write your code here
7       return "";
8   }
9
10  public static void main(String[] args) {
11      System.out.println("Rearranged string: " + RearrangeString.rearrangeString("aappp"));
12      System.out.println("Rearranged string: " + RearrangeString.rearrangeString("Programming"));
13      System.out.println("Rearranged string: " + RearrangeString.rearrangeString("aapa"));
14  }
15  }

```

Run

Save

Reset



Solution

This problem follows the [Top 'K' Numbers](#) pattern. We can follow a greedy approach to find an arrangement of the given string where no two same characters come next to each other.

We can work in a stepwise fashion to create a string with all characters from the input string. Following a greedy approach, we should first append the most frequent characters to the output strings, for which we can use a **Max Heap**. By appending the most frequent character first, we have the best chance to find a string where no two same characters come next to each other.

So in each step, we should append one occurrence of the highest frequency character to the output string. We will not put this character back in the heap to ensure that no two same characters are adjacent to each other. In the next step, we should process the next most frequent character from the heap in the same way and then, at the end of this step, insert the character from the previous step back to the heap after decrementing its frequency.

Following this algorithm, if we can append all the characters from the input string to the output string, we would have successfully found an arrangement of the given string where no two same characters appeared adjacent to each other.

Code

Here is what our algorithm will look like:

Java

Python3

C++

JS

```

1  import java.util.*;
2
3  class RearrangeString {
4
5      public static String rearrangeString(String str) {
6          Map<Character, Integer> charFrequencyMap = new HashMap<>();
7          for (char chr : str.toCharArray())
8              charFrequencyMap.put(chr, charFrequencyMap.getOrDefault(chr, 0) + 1);
9
10         PriorityQueue<Map.Entry<Character, Integer>> maxHeap = new PriorityQueue<Map.Entry<Character, Integer>>(
11             (e1, e2) -> e2.getValue() - e1.getValue());
12
13         // add all characters to the max heap
14         maxHeap.addAll(charFrequencyMap.entrySet());
15
16         Map.Entry<Character, Integer> previousEntry = null;
17         StringBuilder resultString = new StringBuilder(str.length());
18         while (!maxHeap.isEmpty()) {
19             Map.Entry<Character, Integer> currentEntry = maxHeap.poll();
20             // add the previous entry back in the heap if its frequency is greater than zero

```

```
20 // add the previous entry back in the heap if its frequency is greater than zero
21 if (previousEntry != null && previousEntry.getValue() > 0)
22     maxHeap.offer(previousEntry);
23 // append the current character to the result string and decrement its count
24 resultString.append(currentEntry.getKey());
25 currentEntry.setValue(currentEntry.getValue() - 1);
26 previousEntry = currentEntry;
27 }
28
29 // if we were successful in appending all the characters to the result string, return it
30 return resultString.length() == str.length() ? resultString.toString() : "";
31 }
32
33 public static void main(String[] args) {
34     System.out.println("Rearranged string: " + RearrangeString.rearrangeString("aapp"));
35     System.out.println("Rearranged string: " + RearrangeString.rearrangeString("Programming"));
36     System.out.println("Rearranged string: " + RearrangeString.rearrangeString("aapa"));
37 }
38 }
39
```

Run

Save

Reset



Time complexity

The time complexity of the above algorithm is $O(N * \log N)$ where 'N' is the number of characters in the input string.

Space complexity

The space complexity will be $O(N)$, as in the worst case, we need to store all the 'N' characters in the **HashMap**.

[< Back](#)☒ Mark As Completed[Next >](#)

Sum of Elements (medium)

Problem Challenge 1



Problem Challenge 1

We'll cover the following



- Rearrange String K Distance Apart (hard)
- Try it yourself

Rearrange String K Distance Apart (hard)

Given a string and a number 'K', find if the string can be rearranged such that the same characters are at least 'K' distance apart from each other.

Example 1:

Input: "mmp", K=2
Output: "mpm" or "pmp"
Explanation: All same characters are 2 distance apart.

Example 2:

Input: "Programming", K=3
Output: "rgmPrmgiano" or "gmrngmrPoa" or "gmrPagimnor" and a few more
Explanation: All same characters are 3 distance apart.

Example 3:

Input: "aab", K=2
Output: "aba"
Explanation: All same characters are 2 distance apart.

Example 4:

Input: "aappa", K=3
Output: ""
Explanation: We cannot find an arrangement of the string where any two 'a' are 3 distance apart.

Try it yourself

Try solving this question here:



Java



Python3



JS



C++



```
1 import java.util.*;
2
3 class RearrangeStringKDistanceApart {
```




```
4
5 public static String reorganizeString(String str, int k) {
6     // TODO: Write your code here
7     return "";
8 }
9
10 public static void main(String[] args) {
11     System.out.println("Reorganized string: " +
12         RearrangeStringKDistanceApart.reorganizeString("mmp", 2));
13     System.out.println("Reorganized string: " +
14         RearrangeStringKDistanceApart.reorganizeString("Programming", 3));
15     System.out.println("Reorganized string: " +
16         RearrangeStringKDistanceApart.reorganizeString("aab", 2));
17     System.out.println("Reorganized string: " +
18         RearrangeStringKDistanceApart.reorganizeString("aappa", 3));
19 }
20 }
21
```

Run

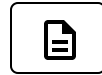
Save

Reset

[< Back](#)☒ Mark As Completed[Next >](#)

Rearrange String (hard)

Solution Review: Problem Challenge 1



Problem Challenge 2

We'll cover the following



- Scheduling Tasks (hard)
- Try it yourself

Scheduling Tasks (hard)

You are given a list of tasks that need to be run, in any order, on a server. Each task will take one CPU interval to execute but once a task has finished, it has a cooling period during which it can't be run again. If the cooling period for all tasks is 'K' intervals, find the minimum number of CPU intervals that the server needs to finish all tasks.

If at any time the server can't execute any task then it must stay idle.

Example 1:

Input: [a, a, a, b, c, c], K=2

Output: 7

Explanation: a -> c -> b -> a -> c -> idle -> a

Example 2:

Input: [a, b, a], K=3

Output: 5

Explanation: a -> b -> idle -> idle -> a



Try it yourself



Try solving this question here:

 Java Python3 JS C++

```
1 import java.util.*;
2
3 class TaskScheduler {
4
5     public static int scheduleTasks(char[] tasks, int k) {
6         // TODO: Write your code here
7         return -1;
8     }
9
10    public static void main(String[] args) {
11        char[] tasks = new char[] { 'a', 'a', 'a', 'b', 'c', 'c' };
12        System.out.println("Minimum intervals needed to execute all tasks: " + TaskSched
13
14        tasks = new char[] { 'a', 'b', 'a' };
15        System.out.println("Minimum intervals needed to execute all tasks: " + TaskSched
16    }
17 }
18
```

Run

Save

Reset

[← Back](#)☒ [Mark As Completed](#)[Next →](#)[Solution Review: Problem Challenge 1](#)[Solution Review: Problem Challenge 2](#)



Problem Challenge 3

We'll cover the following ^

- Frequency Stack (hard)
- Try it yourself

Frequency Stack (hard)

Design a class that simulates a Stack data structure, implementing the following two operations:

1. `push(int num)`: Pushes the number 'num' on the stack.
2. `pop()`: Returns the most frequent number in the stack. If there is a tie, return the number which was pushed later.

Example:

After following push operations: `push(1)`, `push(2)`, `push(3)`, `push(2)`, `push(1)`, `push(2)`, `push(5)`

1. `pop()` should return 2, as it is the most frequent number
2. Next `pop()` should return 1
3. Next `pop()` should return 2

Try it yourself

Try solving this question here:



```
1 import java.util.*;
2
3 class FrequencyStack {
4
5     public void push(int num) {
6         // TODO: Write your code here
7     }
8
9     public int pop() {
10        // TODO: Write your code here
11        return -1;
12    }
13
14    public static void main(String[] args) {
15        FrequencyStack frequencyStack = new FrequencyStack();
16        frequencyStack.push(1);
17        frequencyStack.push(2);
18        frequencyStack.push(3);
19        frequencyStack.push(2);
20        frequencyStack.push(1);
```



```
21 frequencyStack.push(2);  
22 frequencyStack.push(5);  
23 System.out.println(frequencyStack.pop());  
24 System.out.println(frequencyStack.pop());  
25 System.out.println(frequencyStack.pop());  
26 }  
27 }  
28
```

Run

Save

Reset

[← Back](#)☒ [Mark As Completed](#)[Next →](#)[Solution Review: Problem Challenge 2](#)[Solution Review: Problem Challenge 3](#)